Identifying LLMs from Their Responses
CMPSC 448 Midterm Project Report

Ryan Broadhurst (team leader), rhb5261@psu.edu
Michael Finn (team member), maf6533@psu.edu
Code, data and results: https://github.com/x04jemdake4/CMPSC-448-Midterm-Project

October 11, 2026

1 Problem Definition and Dataset Curation

Given a text x (the response, the prompt, or both), predict its source y. The main task predicts the family y, either Claude, GPT or Llama. A harder secondary task predicts the individual model for the family, which we had 2 models per family. The networks output scores s = f(x;W), softmax gives P(Y = k | x) = exp(s_k) / Σ_j exp(s_j), and training minimises the cross-entropy L = −(1/N) Σ_i log P(Y = y_i | x_i). We measure performance with accuracy and macro F1, which come out to be the same because every family has the same amount of responses in the dataset.

Our data source was from AlpacaEval, which is a public benchmark of 805 instruction prompts. In its GitHub repository, we then used only 600 responses in our dataset to make the training time shorter. The answers were generated by the benchmark maintainers by asking prompts to each model and recording the responses. This made it easy to verify the output and labeling.

Our script in src/build_dataset.py downloads the six files below and draws a random sample of 100 prompts, which also keeps all six models' answers to exactly those prompts. The result is data/dataset.csv, which is sorted by one row per response: family, model name, prompt, response, prompt source, prompt id and CV fold. We chose two models per family to cover two sizes of each.

Table 1: Dataset composition: perfectly balanced, 200 responses per family. Prompt sources: OpenAssistant 30, self-instruct 28, helpful-base 17, Koala 14, Vicuna 11.

Family	Model	#responses	median words	mean words
Claude	Claude-3-Opus, Claude-3.5-Sonnet	100 + 100	208 / 195	207 / 218
GPT	GPT-4, GPT-4o	100 + 100	218 / 273	199 / 274
Llama	Llama-3-70B, Llama-3-8B	100 + 100	319 / 306	298 / 289

Family	Model	#responses	median words	mean words
Total	6 models, 100 shared prompts	600		

Design choices and checks. Because every model answers the same prompts, the comparison is paired: class differences come from the models, not from topics. This also makes RQ2 a controlled experiment. We scaled down from all 805 prompts and 5 families (8,050 responses) because training on this data took a very long time. This dataset made it easy to have labeled data that was both complete, and did not contain either private or unfinished information. The models are older than frontier models, but that is fine in this case.

Evaluation protocol. We used a 5-fold cross-validation grouped by prompt. We split the 100 prompts into 5 folds of 20, and all six answers to a prompt are in the same group. In each round, one group is dedicated for testing while the other one is meant for deciding when to stop training based on results. The last three groups are for training. This ensures every response is tested only once. We report the mean ± sd over folds, and combined metrics over all 600 test predictions. Grouping by prompt prevents the leak a row-level split would create, where GPT's answer to a question is in training and Claude's answer to the same question is in test.

2 Models and Training

Pre-processing. Before training we separated the responses into tokens and made all words lowercase so the data is easier to work with. Every new line becomes <nl>, and markdown symbol such as # or ** are kept because it is how a model gives its answer which helps detect its style. In each round the vocabulary is built from the training data, so the test data is unseen. Sequences are shortened to 400 tokens maximum. The input+output view (RQ2) is the prompt (at most 128 tokens), a <sep> token, and the response.

CNN: Our CNN follows the design from Kim and uses convolution and pooling layers from what we learned in class. Each filter learns to detect a short pattern of 3-5 tokens. Max-pooling ensures that every filter's strongest match is kept. We combine the results into one 384 number vector, apply a dropout of 0.5 to reduce overfitting the model, and use a final layer to rate each family.

RNN: A one-layer bidirectional LSTM. It reads every response one token at a time, both forward and backwards using the exact same embeddings as cnn with a dropout of 0.5.. It uses the gates $i_t, f_t, o_t = \sigma(\cdot)$, candidate $q_t = \tanh(\cdot)$, cell $p_t = f_t * p_{t-1} + i_t * q_t$ and output $h_t = o_t * \tanh(p_t)$. Sequences are packed, so the padding is never actually read. We then combined the strongest value of each memory unit across the entire response with the final value from each direction, applied the dropout, and used a final layer to then score each family once again.

Training: Both networks share the same training code src/train.py. A cross entropy loss with Adam optimizer was utilized, which was referenced from Chapter 9. We train on small batches of 32 responses, grouped by similar lengths in order to save time. The gradient size was capped at 5 to keep training stable. We train for up to 40 epochs and stop once validation macro-F1 has

not improved for 8 epochs, so this ensures keeping the best versions for each model. Results are reproducible because each round uses a different fixed random seed. To compare the responses we put TF-IDF word and word-count pairs into logistic regression. Ran the training locally on a macbook, which took about 1 hour total.

3 Results

3.1 RQ1: Can we identify which LLM generated a response?

Yes, from the response alone and well above chance (Table 2).

Table 2: RQ1: classification from the response only. Accuracy is mean ± sd over 5 folds. Macro-F1 is pooled over all 600 out-of-fold predictions.

	3 families (chance 33.3%)		**6 models** (chance 16.7%)	
Classifier	Accuracy	Macro-F1	Accuracy	Macro-F1
TF-IDF + LR (reference)	69.8 ± 3.2%	0.692	43.3 ± 2.7%	0.434
CNN	68.7 ± 3.4%	0.683	42.2 ± 5.0%	0.421
BiLSTM	54.0 ± 3.9%	0.538	29.3 ± 2.7%	0.291

CNN vs. RNN. The CNN reaches 68.7% accuracy on the family task, twice as good as random guessing. It is statistically tied with TF-IDF (69.8%), because the gap is smaller than normal variation between folds. The BiLSTM is clearly weaker at 54.0%. This matches the CNN-vs-RNN trade-off. The cues are mostly local patterns (a "- **" bullet), which the filters are built to spot short patterns like these within the text to detect the family.. The LSTM must carry that evidence through up to 400 sequential steps and learn from only 360 training responses per fold. It learns far more slowly (Figure 1): in the first fold the CNN's training loss falls below 0.1 within 20 epochs, while the LSTM's is still 0.47 after 30 epochs, with validation accuracy around 58%.

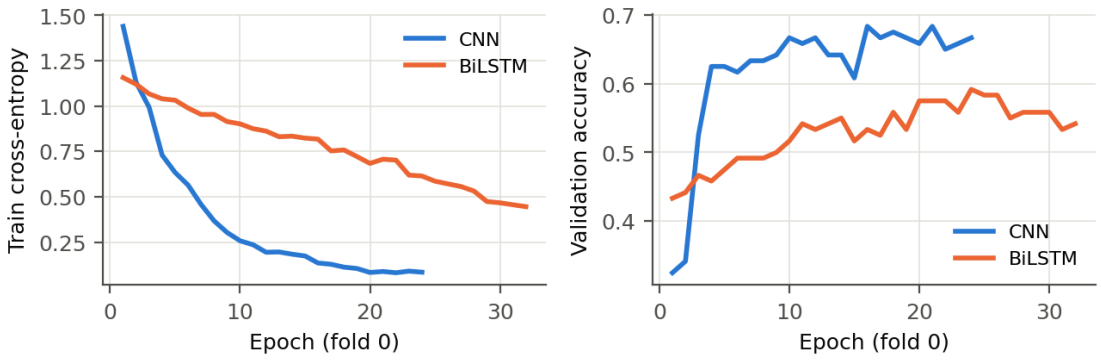


Figure 1: Training curves for fold 0 (family task): training cross-entropy (left) and validation accuracy (right). Early stopping keeps the checkpoint with the best validation macro-F1.

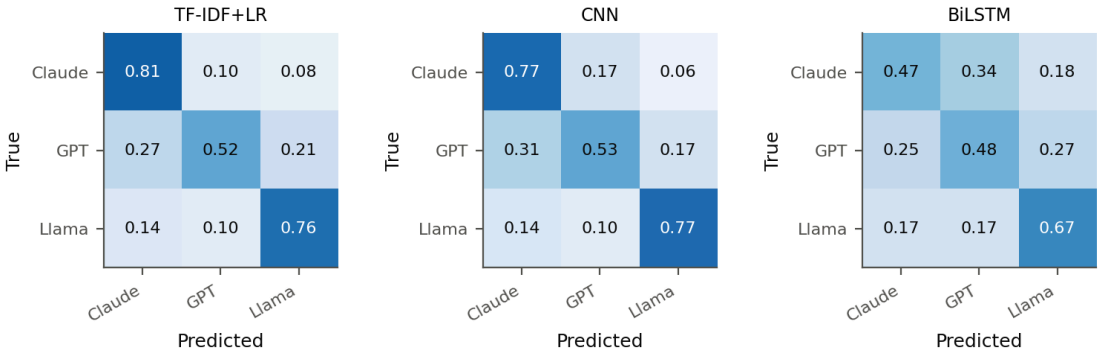


Figure 2: Row-normalised confusion matrices for the family task (pooled over the 5 test folds, 600 responses).

Which families are confused? For the CNN (Figure 2), Claude and Llama are each recognised 77% of the time, but GPT only 52%, and 31% of GPT responses are labelled Claude. The 6-model confusion matrix (in results) explains this. 54% of GPT-4's answers are attributed to a Claude model, while GPT-4o is mistaken for Llama far more often than GPT-4 is (16% vs. 2%). Plain, markdown-free GPT-4 reads like Claude, and markdown-heavy GPT-4o reads closer to Llama (Section 4.2), so the GPT family is split between two styles.

Individual models. Separating the two models within a family is much harder. The CNN reaches 42.2% (about 2.5× chance), TF-IDF 43.3%, and the LSTM only 29.3%. 40% of the CNN's errors are confusions between the two siblings of a family. For example, Llama-3-8B is labelled Llama-3-70B 45% of the time. Models within a family share most of their fingerprint, so the family is the natural unit of attribution.

3.2 RQ2: Does the user's prompt help identify the LLM?

Table 3: RQ2: family accuracy (mean ± sd over 5 folds) for the three input views. Chance is 33.3%.

Classifier	Input (prompt) only	Output only	Input + output
TF-IDF + LR	33.3 ± 0.0%	69.8 ± 3.2%	68.0 ± 1.9%
CNN	33.3 ± 0.0%	68.7 ± 3.4%	67.5 ± 1.5%
BiLSTM	33.3 ± 0.0%	54.0 ± 3.9%	44.7 ± 3.7%

(i) Can the LLM be predicted from the prompt alone? No, because all three of the classifiers scored exactly 33.3% in every fold. This is guaranteed by the design: each prompt appears six times, once for each model. Thus, any prediction for a prompt is bound to be correct for exactly one third of its rows. Therefore, the Bayes-optimal accuracy from the prompt alone is ⅓.

(ii) Does including the prompt help? No, but it can hurt. The combination of input and output is never better than only output and is clearly worse for the BiLSTM. The prompt adds tokens with 0 label information, which act as noise for a smaller training set. It also takes up 128 of the 400 tokens that the network reads, so a good portion at the end of longer responders are cut off. The sequential LSTM suffers most, because they have to pass irrelevant prompt tokens through before the actually informative response can begin.

(iii) Why might input-only classification work surprisingly well elsewhere? Because of selection bias. In typical AI use, users choose which model to ask based on the task they are looking to have completed, so the prompt distribution itself differs by model. A classifier will then learn the topic distribution of each model's users, not the model itself.. We tried to emulate this with a small simulation in which each prompt kept only one of the family's answers under a preference that was domain dependent. However, the bias ended up being too weak to learn from just 100 prompts, as input only accuracy equally that of the majority baseline (39%). Despite this, our conclusion remains the same. If input-only classification beats the majority baseline on some dataset, then that is a measure of an outside influence on how the data was collected, and not of a property of the LLM.

4 In-depth Analyses

4.1 RQ3 (extra credit): Do LLM fingerprints generalize across tasks and domains?

Protocol. The OOD classifier is trained only on prompts outside of each of the held-out targets T and is tested on all responses to prompts within each T. The ID reference is the RQ1 cross-validated prediction for the same rows, made by classifiers that did see other prompts from T. The OOD models are trained on at least as many prompts as ID models (60) so that a drop in accuracy under OOD cannot be blamed on less training data. There are two kinds of targets: task type, as suggested in the assignment, and prompt source.

Table 4: RQ3: 3-family accuracy (%) on each held-out target. ID = classifier trained with other prompts of the target (RQ1 cross-validation); OOD = target never seen in training. Δ = OOD − ID. CNN OOD numbers are the mean of 3 training seeds.

		TF-IDF + LR			CNN		
Held-out target	#prompts	ID	OOD	Δ	ID	OOD	Δ
Task: writing	9	66.7	63.0	-3.7	51.9	61.1	+9.3
Task: tech	10	56.7	60.0	+3.3	60.0	55.0	-5.0
Mean over task types		61.7	61.5	-0.2	55.9	58.1	+2.1
Mean over sources		71.7	71.0	-0.7	70.0	69.4	-0.6

(i) Does performance drop on unseen tasks? No, at least not usually. Holding out a task type increases the accuracy of CNN by 2.1 points, while it decreases TF-IDF by 0.2 points. Holding out a prompt source decreases the accuracy of CNN by -0.6 while it decreased TF-IDF by 0.7 points. Every one of the OOD accuracies were far above chance, staying at or above 55%. **(ii) Are some families more consistently identifiable?** Yes. Averaged out over all held-out targets, the CNN's out of distribution F1 is the highest for Llama at 0.75, and the lowest for GPT at 0.56. Stylization habits that appear in every kind of answer (such as Llama's bold labels and exclamations, and Claude's plain numbered lists) transfer between models within their families the best. The GPT family is the least consistent because its two members write in different styles. **(iii) A general fingerprint or task-specific patterns?** Mostly general from what we saw. A classifier that is task specific would collapse when performing an unseen task, while ours does not. This conclusion/answer agrees with RQ4: the strongest cues from each model are formatting and stylistic habits that are present in any response from the model.

4.2 RQ4 (extra credit): What characteristics distinguish the LLMs?

Stylometric profile. We computed 21 interpretable features per response (Figure 3).

- **Llama-3** writes the longest answers (averaging 283 words vs. ChatGPT's 228 and Claude's 203), uses the most bold (1.5 spans per 100 words, often being list labels), uses 4 times more exclamation marks, is the only family that writes "I hope this helps", and has the lowest lexical diversity (0.746 vs. 0.77).

- **GPT** is defined mostly by GPT-4o. This is the only model using ### headers (5.3% of its lines vs. the other using it ≤0.3% of the time), typographic apostrophes and LaTeX math, and it opens with "Certainly!/Sure!" the most often (13.5% of the time). GPT-4 uses almost no markdown.

- **Claude** uses no markdown bold at all (0 of 200 responses). It prefers plain numbered lists (41% of lines, the highest) separated by blank lines, and writes the shortest sentences (15 words on average).

A classifier, when given only these 21 numbers and with no words at all, reaches 66.5% using logistic regression and 69.7% with random forest. These are as good as the CNN when reading the full text. The random forest relies most on bold density, length, lexical diversity and blank-line ratio. The top TF-IDF n-grams are also mostly formatting: <nl> <nl> and <nl> 1. for Claude, - ** and <nl> ### for GPT, **, ** : and ! for Llama.

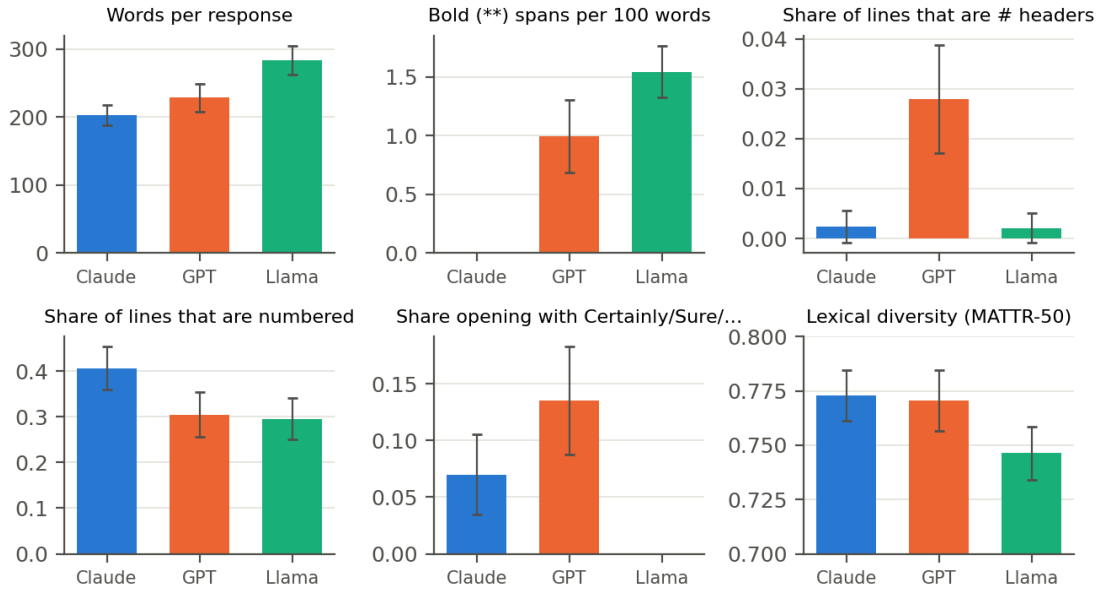


Figure 3: Selected stylometric features by family (means with 95% CIs over 200 responses each).

Ablations. We re-trained both classifiers after removing one kind of signal from every response.

Text given to the classifier (pooled 5-fold CV accuracy)	TF-IDF + LR	CNN
Original response (RQ1)	69.8%	68.7%
First and last line removed (openers such as “Certainly! Here is...”, sign-offs such as “I hope this helps!”)	64.2%	65.0%
All formatting removed (markdown **, #, bullets, numbering, code fences; all line breaks collapsed)	53.5%	52.5%
Both removed (only the wording of the body is left)	46.3%	45.0%
Words randomly shuffled within each response (vocabulary kept, order destroyed)	65.2%	59.2%
Only the first 50 tokens	62.2%	62.3%

Table 5: RQ4 ablations: pooled 5-fold accuracy, 3 families, chance 33.3%. Each row retrains from scratch.

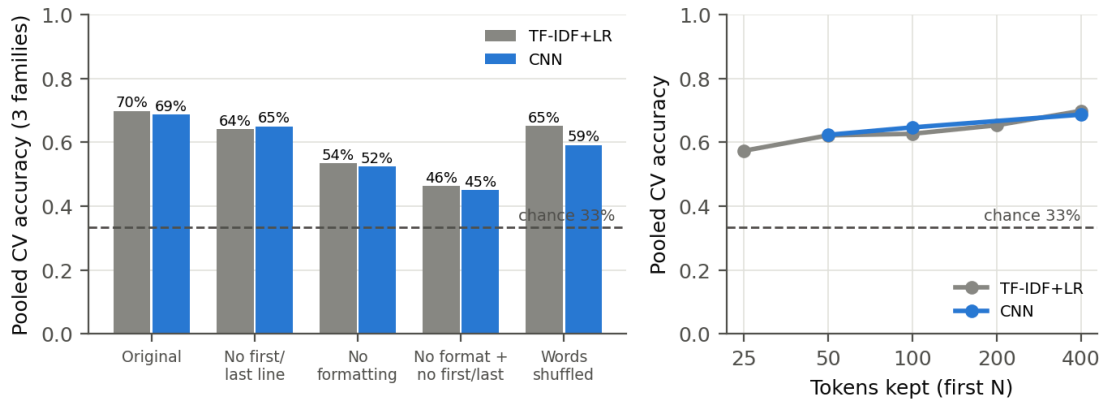


Figure 4: RQ4 ablations. Left: accuracy after removing a signal. Right: accuracy when only the first N tokens are kept.

Answers. (i) What signals does the classifier use? They are mainly presentation: markdown bold and headers, list style, line breaks and length. Following those would be characteristic openers, closers and vocabulary. **(ii) Linguistic, structural, or both?** Both. The largest drop was from removing formatting, which cost about 16 points. And removing openers and sign-offs costs only between 4 and 6 points. With both of them gone, the wording alone still gives 46.3% / 45.0%, which sits far above chance (33.3%). While shuffling the words barely hurts TF-IDF (65.2%), it costs the CNN about 10 points (59.2%), so the CNN's filters do really focus on reading local word order. **(iii) Does removing signals reduce performance substantially?** Yes. Altogether, removing formatting, openers, and closers costs about 23 points. Still, the removal of none of them alone brings accuracy to chance, so the fingerprint is spread over several redundant cues. This trend also appears early, with the first 50 tokens alone giving about 90% of the full-text accuracy.

5 Lessons and Experience

1. The evaluation protocol matters as much as the model. Choosing to split by prompt over splitting by row as an important decision. With the data being paired, a row split would put one model's answer to a question in training while another model's answer to the same question would be put in test - inflating scores. Five-fold CV on only 600 examples gave every response one honest prediction and a standard deviation for every number. Getting values like these allow us to make supported claims like "the CNN and TF-IDF are tied".

2. Always compare against a simple baseline. On its own, 69% CNN accuracy looks like deep learning at work. The TF-IDF baseline matched it, and 21 hand-crafted style features matched it too. Checking what the cheapest model achieves before crediting a neural network was our most useful habit.

3. Match the inductive bias to the signal. Convolution with max-pooling suits local, position-independent cues such as list markers and stock phrases. The LSTM's sequential

memory is not needed for these, cannot parallelise over time steps (the “slow, sequential” drawback from lecture), and learned poorly from 360 training examples per fold. With more data or pretrained embeddings the gap should shrink, and a pretrained Transformer encoder (Ch. 6) is the natural next model.

4. Budget compute before scaling up. Our first design was looking to create a dataset way too large for our scope for this project (805 prompts, 5 families, 8,050 responses) and needed about 2 minutes per BiLSTM epoch on a 2-core cloud CPU. We realised this and cut down the scope to 3 families and 100 prompts, switched to cross validation to keep the estimates reasonable, and started running locally on a laptop. Making the code reproducible (fixed seeds, one run_all.py, automatic GPU use) paid off: the re-run on a second machine reproduced TF-IDF exactly and the CNN to within 0.1 points. The cost is lower power for more exact analysis (some of the RQ3 targets have only 9–11 prompts).

5. Fingerprints are partly packaging, and data design decides RQ2. A lot of what separates these models is formatting, which is removed when a user is asking for plain text. Therefore, using a robust detector should be tested with formatting stripped. And because every model answered the same prompts, classification based only off of prompts becomes a possibility. When dealing with uncontrolled data, a high score that is only input should be read as a warning sign of selection bias.

References

1. X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, T. B. Hashimoto. *AlpacaEval: An Automatic Evaluator of Instruction-following Models*. GitHub repository, 2023. https://github.com/tatsu-lab/alpaca_eval

2. Y. Dubois, B. Galambosi, P. Liang, T. B. Hashimoto. Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. arXiv:2404.04475, 2024.

3. Y. Kim. Convolutional Neural Networks for Sentence Classification. *EMNLP*, 2014.

4. S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. *Neural Computation* 9(8), 1997.

5. D. P. Kingma, J. Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.